

Virtual ECU / Socket Co-Simulation Demo

A small, self-contained demo of the pre-silicon validation pattern used in production automotive firmware work: ****run real firmware against a fully simulated MCU**, and let an external process talk to it over a socket instead of a physical wire.**

This is a scaled-down, public-target rebuild of an architecture I built professionally on an Infineon TRAVEO T2G virtual ECU – a QEMU-based platform with a custom SPI peripheral model (register-level behavior, interrupts, DMA/FIFO support) and a socket-based backend so the virtual ECU could talk to external Python hardware simulators, used to validate an AUTOSAR SPI driver before hardware was available. No proprietary code or configuration is reused here – this rebuilds the same **technique** from scratch on public tooling and a public target.

What it does

1. [Renode](https://renode.io/) (an open-source MCU/SoC simulator) boots a real Zephyr RTOS firmware image (a CAN-networking sample) on a simulated ST Nucleo-H753ZI board – a real automotive-relevant MCU target, fully emulated, no hardware attached.
2. The firmware's CAN controller (``fdcan1``) is wired to a virtual CAN hub inside the simulation – the same mechanism used to build multi-node CAN networks entirely in software.
3. The firmware's UART is exposed over a ****TCP socket**** (``emulation CreateServerSocketTerminal``), the co-simulation seam.
4. An external Python script (``hardware_simulator.py``) connects to that socket like a real HIL test rig would talk to a board over a physical UART/SPI link, and streams the firmware's live log output.

See [``sample_run_output.log``](sample_run_output.log) for a captured run: the Zephyr boot banner, CAN MCAN driver initialization, and periodic CAN frame transmission, all observed from **outside** the simulation over a plain TCP socket.

Why this matters for embedded/automotive teams

Waiting on physical boards or silicon is one of the biggest bottlenecks in firmware schedules. A virtual ECU environment like this lets driver and diagnostic software be written, exercised, and partially validated before hardware exists – and lets test infrastructure (CI, external test rigs, protocol analyzers) hook into the simulation exactly as it would hook into real hardware, via a socket instead of a wire.

Running it

Requirements: [Renode](https://renode.io/) installed, Python 3.

```
``powershell
# Terminal 1 - start the simulated ECU
"C:\Program Files\Renode\bin\Renode.exe" --disable-gui virtual_ecu_demo.resc

# Terminal 2 - connect the external "hardware simulator"
python hardware_simulator.py 127.0.0.1 3456 15
````
```

You'll see the Zephyr boot log and periodic CAN frame transmissions stream into Terminal 2, sent entirely over the TCP socket from the simulated firmware.

### ## Two-node virtual CAN network

``virtual_ecu_twonode_demo.resc`` extends the same idea to two independently simulated Nucleo-H753ZI boards ("ecu\_node" and "tester\_node"), each running the same Zephyr firmware, both attached to the same virtual CAN hub. Each node's UART is exposed on its own socket (3456 and 3457), so an external Python script (``dual_node_monitor.py``) can watch both sides of the exchange at once – the same setup used to validate ECU-to-tester communication on a real CAN bench, running entirely in software.

```
``powershell
```

```
73 "C:\Program Files\Renode\bin\Renode.exe" --disable-gui virtual_ecu_twonode_demo.resc
74 python dual_node_monitor.py 10
75 ```
76
77 [twonode_sample_output.log] (twonode_sample_output.log) is a captured run:
78 each node transmits an 8-byte CAN frame (ID 0x1) and receives the frame the
79 other node sent, confirmed by matching payload bytes observed
80 independently on both sockets – real firmware, on two independently
81 simulated MCUs, exchanging real CAN frames over a shared virtual bus.
82
83 ## Files
84
85 - virtual_ecu_demo.resc – single-node demo: firmware + CAN hub + UART
86 exposed over a socket.
87 - hardware_simulator.py – external co-simulation client for the
88 single-node demo.
89 - sample_run_output.log – a captured single-node run.
90 - virtual_ecu_twonode_demo.resc – two-node demo: two simulated ECUs
91 sharing one virtual CAN bus.
92 - dual_node_monitor.py – external client watching both nodes at once.
93 - twonode_sample_output.log – a captured two-node run.
94
95 ## Possible extensions
96
97 - Replace the periodic CAN sample traffic with a minimal UDS diagnostic
98 exchange (ReadDataByIdentifier / DTC read-clear) between the two nodes,
99 mirroring real ECU-to-tester diagnostic validation.
100 - Replace the UART-socket seam with a custom register-level peripheral
101 (the SPI-style pattern from the original work) using Renode's C#
102 peripheral framework.
103
104 ---
105
106 *Built as a portfolio piece demonstrating pre-silicon validation tooling –
107 happy to discuss adapting this pattern to your target MCU/RTOS.*
108
```